Plataforma de Emprestimos P2P

Documentacao Tecnica — V1

Arquitetura Geral + Schema do Banco de Dados

Gerado em 26/05/2026

STACK BACK-END
NestJS + TypeScript

BANCO DE DADOS
PostgreSQL + Prisma ORM

FILAS & CACHE
Redis + BullMQ

AUTOMACAO
n8n + Webhooks

FRONT-END
Next.js + Tailwind + shadcn

INFRAESTRUTURA
Docker + Docker Compose

1. Visao Geral do Projeto

A V1 da plataforma e um sistema operacional inteligente para analise, aprovacao e gerenciamento de emprestimos P2P (peer-to-peer lending). O dinheiro real permanece na conta do administrador; o sistema realiza controle logico, operacional e financeiro visual.

Objetivos da V1

- Sistema de emprestimos com investidores e clientes
- Score automatico baseado em formulario e comportamento
- Aprovacao hibrida: automatica + confirmacao do ADM
- Controle visual de saldo e investimentos
- Automatizacoes financeiras e operacionais via n8n

Tipos de Usuario

Perfil	Responsabilidade	Dashboards
Cliente	Solicita emprestimos e realiza pagamentos	Solicitacoes, parcelas, pagamentos
Investidor	Adiciona saldo visual e acompanha rendimentos	Carteira, investimentos, rendimentos
Administrador	Aprova operacoes e monitora toda a plataforma	Visao geral, aprovacoes, auditoria

2. Arquitetura Geral

O sistema e organizado em cinco camadas independentes que se comunicam via API REST (JWT) e mensageria assincrona (BullMQ/Redis). Cada camada pode ser escalonada ou substituida sem impactar as demais.

Camada 1 — Front-end

Next.js + Tailwind CSS + shadcn/ui

- Tres dashboards distintos conforme o role do usuario (Cliente, Investidor, ADM)
- Autenticacao via JWT com refresh token (gerenciado em cookie httpOnly)
- Upload de documentos e selfie diretamente para o backend
- Notificacoes em tempo real via polling ou WebSocket (futuro)

Camada 2 — Back-end (API)

NestJS + TypeScript — organizado em modulos independentes

- auth/ — JWT, refresh token, guards de role, decorators
- users/ — cadastro, KYC, perfil, score
- loans/ — solicitacao, aprovacao, geracao de parcelas
- payments/ — PIX, QR Code, webhook de confirmacao
- score/ — calculo automatico baseado em eventos
- notifications/ — disparos por e-mail e in-app
- storage/ — upload/download de documentos no S3
- prisma/ — PrismaService global injetado em todos os modulos
- common/ — DTOs, pipes, filtros de excecao globais

Camada 3 — Dados & Cache

PostgreSQL + Prisma ORM + Redis + BullMQ + Storage S3

- PostgreSQL: fonte de verdade de todos os dados transacionais
- Prisma ORM: type-safe, migrations versionadas, schema como codigo
- Redis: cache de sessoes, rate limiting e pub/sub
- BullMQ: filas de jobs (recalculo de score, notificacoes, retry de webhook)
- S3 (ou MinIO local): armazenamento de documentos e QR Codes

Camada 4 — Automacao

n8n (low-code) orquestrando fluxos automatizados via webhook

- Atualizacao automatica de parcelas pagas apos confirmacao do webhook
- Recalculo dinamico de score a cada evento relevante
- Envio de notificacoes de atraso (D-1, D+1, D+3, D+7)
- Alertas automaticos para o ADM em casos criticos
- Integracao com servicos externos de e-mail e WhatsApp (futuro)

Camada 5 — Infraestrutura

Docker + Docker Compose — todos os servicos containerizados

- Um docker-compose.yml sobe todo o ambiente: API, DB, Redis, n8n
- Variaveis de ambiente via .env (nunca hardcoded no codigo)
- Volumes persistentes para PostgreSQL e Redis
- Healthchecks em todos os servicos criticos
- Preparado para CI/CD e deploy em VPS ou cloud (futuro)

Fluxo de uma Requisicao Autenticada

Toda requisicao protegida segue este caminho dentro do back-end:

1. HTTP Request chega no Controller (rota publica ou protegida)
2. JwtAuthGuard valida o token — se invalido, retorna 401
3. RolesGuard verifica se o role do usuario tem permissao — se nao, retorna 403
4. ValidationPipe valida o DTO da requisicao com class-validator
5. Controller chama o Service correspondente
6. Service executa a logica de negocio e acessa o banco via PrismaService
7. Response e retornado ao cliente em formato JSON padronizado

Fora do escopo da V1

- Carteira bancaria real ou banco digital
- PIX automatico de saida (transferencias programadas)
- Split financeiro real entre investidores
- IA complexa de credito (score e baseado em regras)
- Microservicos (monolito modular ate validar o negocio)
- App mobile nativo

3. Stack Tecnologica

Camada	Tecnologia	Justificativa
Back-end	NestJS + TypeScript	Framework opinado, modular, com DI nativo — ideal para APIs escaláveis
Banco de Dados	PostgreSQL	ACID, suporte a JSON, excelente para dados financeiros transacionais
ORM	Prisma ORM	Type-safe, migrations versionadas, DX excelente com TypeScript
Front-end	Next.js	SSR/SSG, roteamento por arquivo, facil deploy e SEO
UI	Tailwind + shadcn/ui	Design system consistente sem CSS custom excessivo
Filas	Redis + BullMQ	Jobs assincronos, retry automatico, painel de monitoramento
Automacao	n8n	Low-code para fluxos: sem precisar de dev para alterar regras de negocio
Auth	JWT + Refresh Token	Stateless, seguro, com rotacao de token e invalidacao por hash
Infra	Docker + Compose	Ambiente identico entre dev e producao, facil onboarding
Storage	S3 / MinIO	Armazenamento escalavel de documentos, URLs assinadas para acesso segu

4. Banco de Dados — Visao Geral

O schema foi projetado com quatro principios fundamentais:

- IDs UUID v4 em todas as tabelas — mais seguro e escalavel que auto-increment sequencial
- Campos createdAt/updatedAt em todos os models — auditoria basica sem custo
- Snapshot de dados criticos no momento do evento (ex: scoreAtRequest em Loan) — historico confiavel
- Campos [FUTURO] comentados no schema — prontos para ativar sem migration destrutiva

Relacionamentos Principais

De	Para	Cardinalidade	Motivo
User	UserProfile	1:1	Separacao auth vs dados pessoais
User	Document	1:N	Multiplos documentos por usuario
User	InvestorWallet	1:0..1	Apenas investidores tem carteira
User	Loan	1:N	Um cliente pode ter varios emprestimos
Loan	LoanInvestment	1:N	Multiplos investidores por emprestimo
Loan	Installment	1:N	Parcelas geradas na aprovacao
Installment	Payment	1:N	Cada parcela pode ter multiplas tentativas de pagamento
InvestorWallet	WalletTransaction	1:N	Extrato imutavel de movimentacoes

5. Schema Detalhado — Models

Model: User

Tabela central de autenticacao. Separada do perfil para agilizar queries de auth e permitir multiplos perfis no futuro (ex: mesmo login para PF e PJ).

Campo	Tipo	Descricao
`id`	`UUID (PK)`	Identificador unico universal
`email`	`String UNIQUE`	Email de login, indexado
`passwordHash`	`String`	Senha em bcrypt — nunca texto puro
`role`	`Role (enum)`	CLIENT | INVESTOR | ADMIN
`status`	`UserStatus`	PENDING_VERIFICATION | ACTIVE | SUSPENDED | BLOCKED
`refreshTokenHash`	`String?`	Hash do refresh token para rotacao segura de sessao
`createdAt`	`DateTime`	Data de criacao do registro
`updatedAt`	`DateTime`	Data da ultima atualizacao

Campos previstos para versoes futuras:

[FUTURO] 2FA via TOTP (twoFactorSecret)

[FUTURO] Login social (OAuth Google/Apple)

Model: UserProfile

Dados pessoais e de score do usuario. Separado de User para manter a tabela de auth enxuta.

Campo	Tipo	Descricao
`fullName`	`String`	Nome completo
`cpf`	`String UNIQUE`	CPF indexado e unico
`phone`	`String`	Telefone para contato e notificacoes
`birthDate`	`DateTime?`	Data de nascimento
`zipCode..state`	`String?`	Endereco separado em campos para validacao por CEP
`scoreValue`	`Int`	Score interno 0-1000, atualizado automaticamente
`kycStatus`	`KycStatus`	PENDING | APPROVED | REJECTED

Campos previstos para versoes futuras:

[FUTURO] Score externo (Serasa/SPC)

[FUTURO] Dados bancarios para PIX de saida

[FUTURO] Re-verificacao periodica de KYC

Model: Document

Documentos enviados pelo usuario para o processo de KYC. Cada documento tem status individual.

Campo	Tipo	Descricao
`type`	`DocumentType`	CPF | RG | CNH | PROOF_OF_INCOME | PROOF_OF_ADDRESS | SELFIE
`fileUrl`	`String`	URL do arquivo no S3 (acesso via URL assinada)
`status`	`DocStatus`	PENDING | APPROVED | REJECTED
`rejectionReason`	`String?`	Motivo do rejeito preenchido pelo ADM
`reviewedBy`	`String?`	ID do admin que revisou
`reviewedAt`	`DateTime?`	Data da revisao

Campos previstos para versoes futuras:

[FUTURO] OCR automatico do documento (ocrData Json)

[FUTURO] Validade de documentos com re-upload periodico

Model: InvestorWallet

Saldo visual do investidor. O dinheiro real fica no banco do ADM; este model e o controle logico.

Campo	Tipo	Descricao
`availableBalance`	`Decimal(15,2)`	Saldo livre para novos investimentos
`investedBalance`	`Decimal(15,2)`	Total alocado em emprestimos ativos
`totalEarnings`	`Decimal(15,2)`	Rendimentos acumulados desde o inicio

Campos previstos para versoes futuras:

[FUTURO] withdrawalBalance — saldo liberado para saque automatico

Model: WalletTransaction

Extrato imutavel de todas as movimentacoes da carteira. Nunca deletar registros desta tabela. Permite recalcular o saldo do zero se necessario.

Campo	Tipo	Descricao
`type`	`String`	CREDIT (entrada) ou DEBIT (saida)
`amount`	`Decimal(15,2)`	Valor da movimentacao
`description`	`String`	Descricao legivel: ex: 'Investimento no emprestimo #XYZ'
`referenceId`	`String?`	ID do Loan, Payment etc. que gerou a transacao

Campos previstos para versoes futuras:

[FUTURO] status — para transacoes pendentes de confirmacao bancaria

Model: Loan

Emprestimo solicitado pelo cliente. Passa por um ciclo de status desde o rascunho ate a quitacao.

Campo	Tipo	Descricao

`borrowerId`	`UUID (FK)`	Referencia ao usuario tomador
`amount`	`Decimal(15,2)`	Valor solicitado
`interestRate`	`Decimal(5,4)`	Taxa mensal: ex: 0.0350 = 3,50% ao mes
`termMonths`	`Int`	Prazo em meses
`status`	`LoanStatus`	DRAFT > PENDING_SCORE > PENDING_APPROVAL > APPROVED > FUNDED > ACTIVE > PAID
`scoreAtRequest`	`Int?`	Snapshot do score no momento da solicitacao — imutavel
`purpose`	`String?`	Finalidade do emprestimo (influencia o score)
`approvedBy`	`String?`	ID do admin que aprovou
`rejectionReason`	`String?`	Motivo do rejeite

Campos previstos para versoes futuras:
[FUTURO] originalLoanId — referencia para emprestimos renegociados
[FUTURO] Garantias (colateral) com tipo e valor

Model: LoanInvestment

Relacionamento entre um emprestimo e seus investidores. Um emprestimo pode ter funding de multiplos investidores.

Campo	Tipo	Descricao
`loanId`	`UUID (FK)`	Emprestimo financiado
`investorId`	`UUID (FK)`	Investidor que aportou
`amount`	`Decimal(15,2)`	Valor aportado por este investidor
`expectedReturn`	`Decimal(15,2)`	Retorno esperado ao final do prazo
`actualReturn`	`Decimal(15,2)`	Retorno ja recebido (atualizado a cada parcela paga)

Campos previstos para versoes futuras:
[FUTURO] isForSale / salePrice — mercado secundario de cotas entre investidores

Model: Installment

Parcelas geradas automaticamente pelo LoanService no momento da aprovacao do emprestimo.

Campo	Tipo	Descricao
`number`	`Int`	Numero da parcela: 1, 2, 3 ... N
`amount`	`Decimal(15,2)`	Valor original da parcela
`dueDate`	`DateTime`	Data de vencimento — usada pelo job de atraso no n8n
`status`	`InstallStatus`	PENDING | OVERDUE | PAID

`fineAmount`	`Decimal(15,2)`	Multa calculada dinamicamente em caso de atraso
`interestAmount`	`Decimal(15,2)`	Juros de mora calculados dinamicamente
`paidAt`	`DateTime?`	Data do pagamento confirmado
`paidAmount`	`Decimal?`	Valor efetivamente pago (pode incluir multa + juros)

Campos previstos para versoes futuras:
[FUTURO] RENEGOTIATED — parcela incluida em renegociacao de divida

Model: Payment

Cada tentativa de pagamento de uma parcela. Um Payment com status CONFIRMED dispara a atualizacao do Installment para PAID.

Campo	Tipo	Descricao
`installmentId`	`UUID (FK)`	Parcela sendo paga
`amount`	`Decimal(15,2)`	Valor do pagamento
`method`	`PaymentMethod`	PIX (outros no futuro)
`status`	`PaymentStatus`	PENDING | CONFIRMED | FAILED | REFUNDED
`pixKey`	`String?`	Chave PIX do ADM (destino)
`qrCode`	`String?`	QR Code em base64 ou URL
`txId`	`String?`	ID unico da transacao no gateway — indexado
`expiresAt`	`DateTime?`	Expiracao do QR Code
`confirmedAt`	`DateTime?`	Timestamp da confirmacao via webhook
`webhookPayload`	`Json?`	Payload completo do webhook para auditoria

Campos previstos para versoes futuras:
[FUTURO] BOLETO, CREDIT_CARD, DEBIT_ACCOUNT como metodos futuros
[FUTURO] REFUNDED — estorno automatico

Model: Notification

Notificacoes enviadas aos usuarios. Suporta multiplos canais e rastreia leitura.

Campo	Tipo	Descricao
`type`	`NotifType`	LOAN_APPROVED | INSTALLMENT_DUE | PAYMENT_CONFIRMED | ...
`channel`	`NotifChannel`	IN_APP | EMAIL (WHATSAPP/SMS/PUSH no futuro)
`title`	`String`	Titulo da notificacao
`body`	`String`	Corpo da mensagem
`read`	`Boolean`	Se o usuario ja leu — false por padrao
`referenceId`	`String?`	ID do objeto relacionado (ex: ID do emprestimo)

`referenceType`	`String?`	Tipo do objeto: 'Loan', 'Payment', 'Installment'

Campos previstos para versoes futuras:

[FUTURO] SYSTEM_ALERT — alertas de manutencao ou avisos gerais

[FUTURO] Push notification para app mobile

Model: AuditLog

Registro imutavel de todas as acoes relevantes do sistema. NUNCA deletar registros desta tabela. Essencial para compliance, suporte e investigacao de inconsistencias.

Campo	Tipo	Descricao
`actorId`	`UUID (FK)`	Quem realizou a acao
`action`	`String`	Ex: LOAN_APPROVED, DOCUMENT_REJECTED, USER_BLOCKED
`entity`	`String`	Entidade afetada: 'Loan', 'Document', 'User'
`entityId`	`String`	ID do registro afetado
`payload`	`Json?`	Snapshot dos dados relevantes no momento da acao
`ipAddress`	`String?`	IP de origem da requisicao
`userAgent`	`String?`	Navegador/cliente de origem

6. Enums do Sistema

Todos os estados possiveis do sistema sao definidos como enums no schema Prisma. Isso garante type-safety em todo o codigo TypeScript e evita strings 'magicas' espalhadas.

Role

- CLIENT — tomador de emprestimo
- INVESTOR — investidor P2P
- ADMIN — administrador da plataforma
- [FUTURO] ANALYST — revisor com acesso parcial

UserStatus

- PENDING_VERIFICATION — cadastrado, aguardando KYC
- ACTIVE — verificado e ativo
- SUSPENDED — suspenso por inadimplencia ou revisao
- BLOCKED — bloqueado permanentemente

KycStatus

- PENDING — documentos enviados, aguardando analise
- APPROVED — KYC aprovado
- REJECTED — KYC reprovado (usuario pode reenviar)
- [FUTURO] EXPIRED — re-verificacao periodica

LoanStatus

- DRAFT
- PENDING_SCORE
- PENDING_APPROVAL
- APPROVED
- FUNDED
- ACTIVE
- PAID
- DEFAULTED
- REJECTED
- CANCELLED
- [FUTURO] RENEGOTIATED

InstallmentStatus

- PENDING — ainda nao venceu
- OVERDUE — vencida e nao paga
- PAID — paga
- [FUTURO] RENEGOTIATED — incluida em renegociacao

PaymentMethod

- PIX
- [FUTURO] BOLETO
- [FUTURO] CREDIT_CARD
- [FUTURO] DEBIT_ACCOUNT

PaymentStatus

- PENDING — aguardando confirmacao
- CONFIRMED — webhook confirmou
- FAILED — PIX expirado ou erro
- [FUTURO] REFUNDED — estornado

NotificationType

- LOAN_APPROVED / LOAN_REJECTED
- INSTALLMENT_DUE / INSTALLMENT_OVERDUE
- PAYMENT_CONFIRMED
- INVESTMENT_RETURN
- KYC_APPROVED / KYC_REJECTED
- [FUTURO] SYSTEM_ALERT

NotificationChannel

- IN_APP
- EMAIL
- [FUTURO] WHATSAPP / SMS / PUSH

7. Models Planejados para Versoes Futuras

Os models abaixo ja estao documentados e parcialmente escritos no schema.prisma (como comentarios). Para ativa-los, basta descomentar e rodar uma migration — sem nenhuma alteracao destrutiva nas tabelas existentes.

[FUTURO] ScoreHistory

Historico de variacao do score de credito de cada usuario.

Ativa quando o produto precisar mostrar a evolucao do score ao cliente ou gerar relatorios de credito. Cada evento que muda o score (pagamento em dia, atraso, novo emprestimo) gera um registro.

- userId — usuario afetado
- oldValue / newValue — score antes e depois
- reason — ex: PAYMENT_ON_TIME, INSTALLMENT_OVERDUE
- referenceId — ID do evento causador

[FUTURO] WithdrawalRequest

Solicitacoes de saque do saldo disponivel pelo investidor.

Ativa quando o produto oferecer PIX de saida. O ADM aprova manualmente (ou automaticamente) e realiza a transferencia fora do sistema. O model registra o pedido e o status.

- investorId — quem solicitou
- amount — valor do saque
- pixKey — chave PIX de destino
- status — PENDING | APPROVED | REJECTED | COMPLETED
- processedBy / processedAt — quem e quando processou

[FUTURO] PlatformConfig

Configuracoes dinamicas da plataforma editaveis pelo ADM sem necessidade de deploy.

Exemplos de chaves: MIN_LOAN_AMOUNT, MAX_LOAN_AMOUNT, MAX_TERM_MONTHS, DEFAULT_INTEREST_RATE, FINE_RATE_PER_DAY. Substitui valores hardcoded no codigo.

- key — nome da configuracao (unico)
- value — valor em string (convertido no codigo)
- description — explicacao para o ADM
- updatedBy / updatedAt — rastreabilidade

8. Proximos Passos Recomendados

1 Configurar o ambiente Docker

Criar docker-compose.yml com PostgreSQL, Redis e n8n. Validar que todos os servicos sobem corretamente antes de escrever qualquer codigo.

2 Inicializar o projeto NestJS

Criar o projeto com @nestjs/cli, configurar o Prisma, rodar a primeira migration e validar a conexao com o banco.

3 Implementar Auth (JWT + Refresh Token)

Modulo auth/ com registro, login, refresh de token e guards de role. E a base de tudo — nada mais deve ser construido antes disso estar solido.

4 Modulo de Usuarios + KYC

Cadastro de clientes e investidores, upload de documentos para S3, fluxo de aprovacao de KYC pelo ADM.

5 Modulo de Emprestimos + Score

Solicitacao, calculo automatico de score, aprovacao hibrida (auto + ADM), geracao de parcelas.

6 Modulo de Pagamentos (PIX + Webhook)

Geracao de QR Code, endpoint de webhook para confirmacao, atualizacao automatica de parcelas.

7 Dashboard ADM + Notificacoes

Painel administrativo, notificacoes in-app e e-mail, integracao com n8n para automacoes.

8 Testes e preparacao para producao

Testes unitarios nos Services, testes e2e nas rotas criticas, configuracao de CI/CD.